CPE 439 Lecture 03

Review

RTOS Ticks

RTOS Tasks Intro

Operating Systems

- RTOS: an operating system designed specifically for in use in real-time systems
- Real-Time System: a system in which correct operation depends on both logical outcomes and the time when the outcome is produced

- Our starting point:
  - Resource constrained environment
  - Single processor
  - Many tasks

The RTOS

- Provides a set of software tools to help process events in real time (if possible)
  - Provides opportunity for fast responses
  - Gives appearance of multitasking
- Real-time system: a system where the response to a given event is deterministic
  - A real-time operating system is an OS that provides a deterministic way of executing the code associated with tasks.
  - A non-deterministic system: events are responded to with no time constraints

Determinism

- Refers to the consistency of timing behavior in a system: ensures no timing constraints are violated
  - A deterministic system responds within predictable intervals regardless of system load
  - Roughly refers to reliability of system

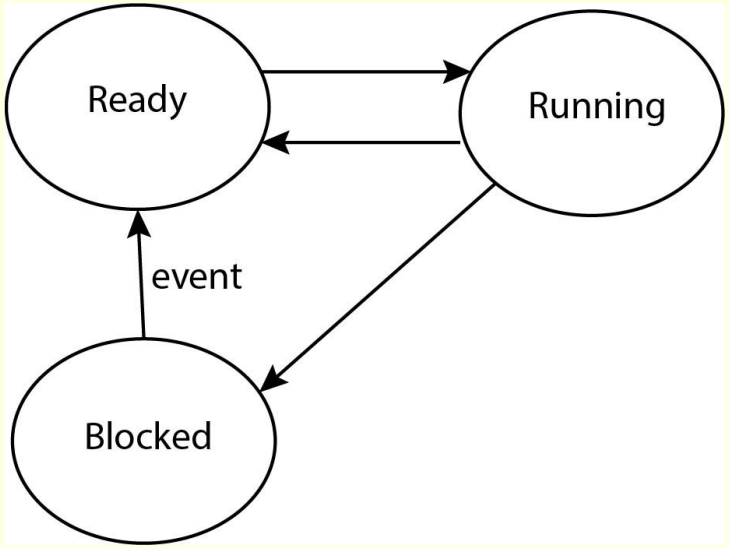
- How to help achieve determinism?
  - Minimizing unbounded delays
  - Use proper scheduling
  - Keeping ISRs short
  - Carefully managing shared resources

The RTOS (in General)

- System is divided into “Tasks”
  - Task: discrete processing unit (code) responsible for implementing task needs
  - Tasks are mini-programs
- Tasks are officially “created”
  - Task creation is a RTOS API call
- Scheduler is started
  - Scheduler takes control and never stops
  - Scheduler is code that executes, but is separate from code in Tasks

Task States

- Tasks can exist in three states
  - Running: task receiving processing time
  - Ready: task has something to do but waiting for higher priority task to complete
  - Blocked: task has nothing to do



- Tasks are not always running
- Task are designed to "block themselves"
- Scheduler responsible for all state changes

Three Important Entities

- The “tick”
  - Give the RTOS a notion of time
- The idle task
  - Ensures RTOS always has some code to execute
- The scheduler
  - Ensures highest priority task is “running”

The RTOS: the Idle Task

- User created tasks aren’t always in running state
  - It’s an MCU, something needs to be running
- Solution: the scheduler creates an idle task
- Idle task:
  - Can go into the running state but does nothing
  - The idle task has a lowest possible priority
    - Allows any other task to always run
    - The idle task runs when no other task is ready to run
      - When all other tasks are “blocking”

Time in an RTOS

- RTOSes: they have their own notion of time
  - RTOS time:
    - Time is subdivided into “ticks”
    - Ticks is a given amount of time
- Tick Implementation:
  - A timer that generates a periodic interrupt
  - Considered the “heartbeat” of the RTOS
  - Creates:
    - a periodic opportunity to “do something”
    - “time slicing” in the RTOS

The RTOS Mechanism

- The scheduler
  - It's code that needs to run
    - If the scheduler is running tasks are not
    - If tasks are running the scheduler is not

- Scheduler "gets to execute" at two instances
  - the only times a context switch can be made

  1) At each tick interrupt
  2) When tasks choose to "block"

RTOS Pseudocode Example

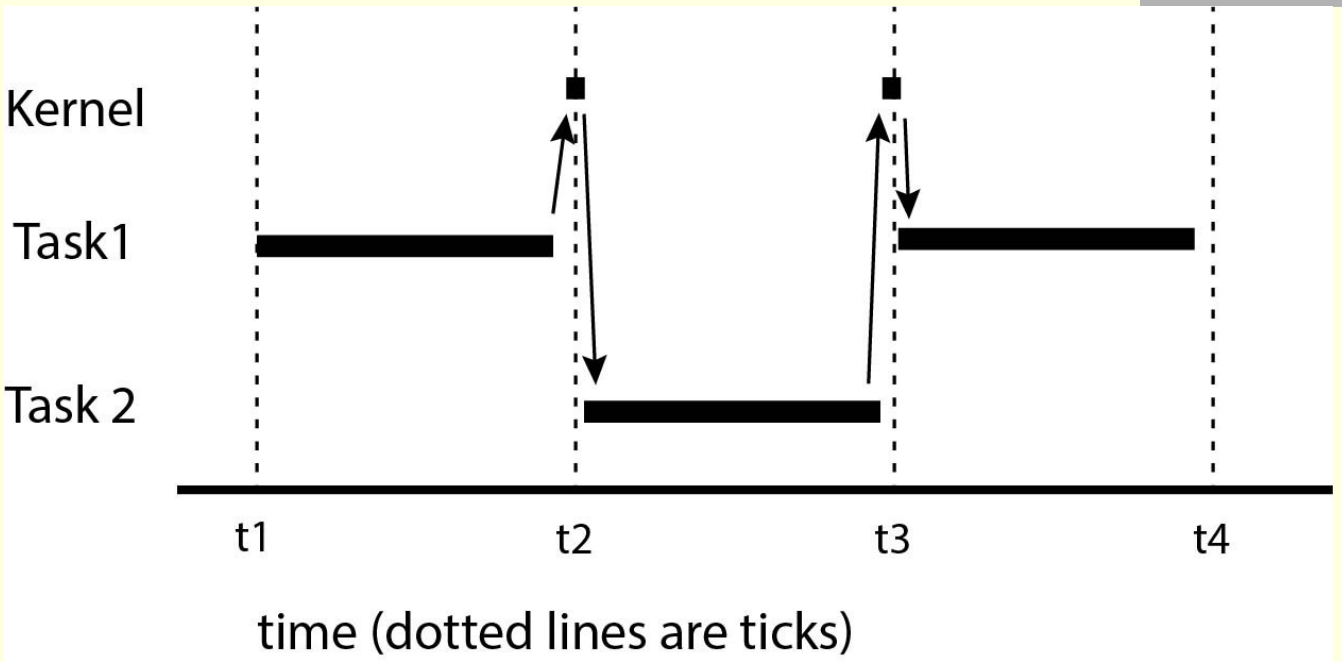
```
void Task1 (parameters)          // continuous processing task
{
   for (;;) {
      // code defining task
   }
}

void Task2 (parameters)         // continuous processing task
{
   for (;;) {
      // code defining task
   }
}

int main (void)
{
   while (1)  {
    Create_task(Task1, priority=2); // two tasks same priority
    Create_task(Task2, priority=2);

    Start_scheduler();
   }
}
```

Example Timing



- Things to note:
  - Scheduler (part of kernel) runs after every tick
  - Tasks have equal priorities:
  - Scheduler implements round robin scheduling to ensure tasks of equal priority run
  - When the kernel is running, the tasks are not

RTOS Pseudocode Example

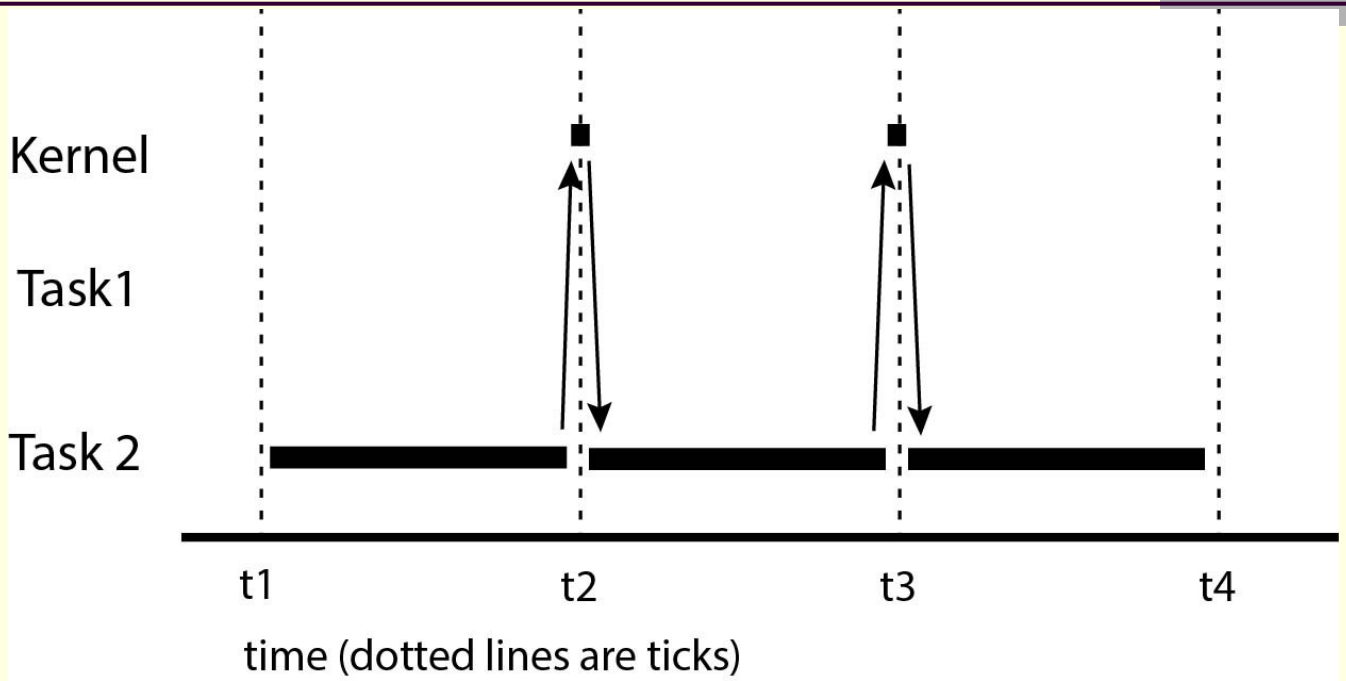
```
void Task1 (parameters)          // continuous processing task
{
   for (;;) {
      // code defining task
   }
}

void Task2 (parameters)         // continuous processing task
{
   for (;;) {
      // code defining task
   }
}

int main (void)
{
   while (1)  {
    Create_task(Task1, priority=1); // lower priority task
    Create_task(Task2, priority=2);

    Start_scheduler();
   }
}
```

Example Timing



- Things to note:
  - Scheduler (part of kernel) runs after every tick
  - Task1has lower priority, thus never “runs”
    - Referred to as “starved”
  - Scheduler keeps highest priority task “running”

Tasks

- Continuous time tasks are not useful
- Tasks generally wait for events
  - Types of events
    - <u>Temporal</u>
      - associated with time
    - <u>Synchronization</u>
      - Based on action of separate task
- The act of a task waiting
  - The task is in the blocked state
  - When event occurs, scheduler moves task from blocked to ready or running state

RTOS Pseudocode Example

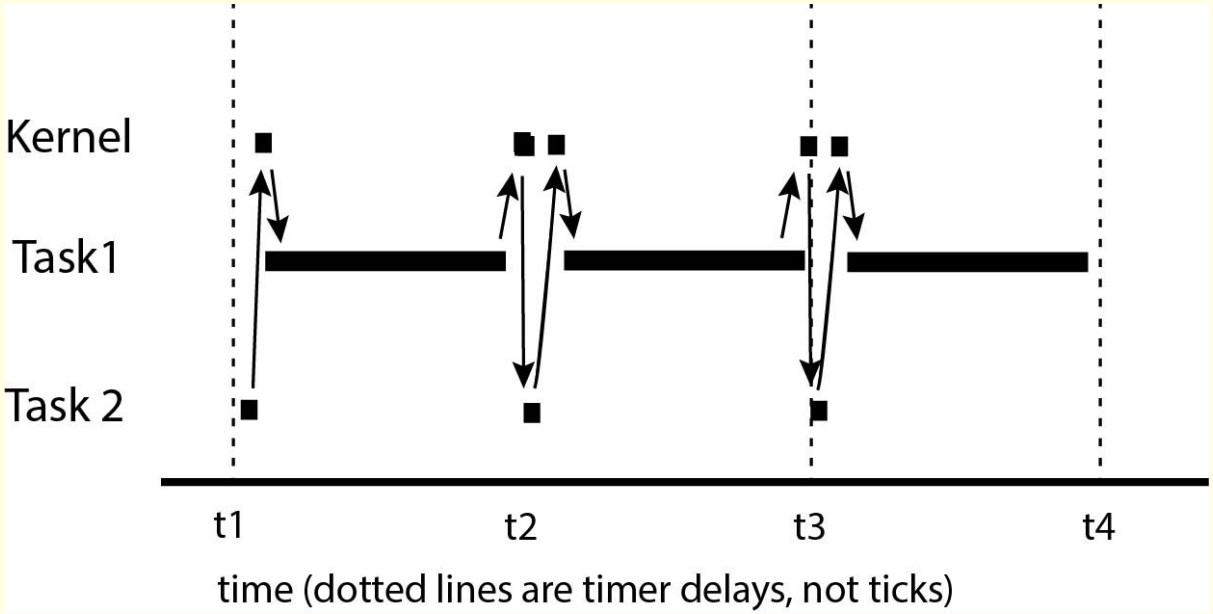
```
void Task1 (parameters)          // continuous processing task
{
   for (;;) {
                                 // code defining task
   }
}

void Task2 (parameters)
{
   for (;;) {
       Delay_API(#_ticks)          // code defining task
   }
}

int main (void)
{
   while (1)  {
    Create_task(Task1, priority=1); // lower priority task
    Create_task(Task2, priority=2);

    Start_scheduler();
   }
}
```

Example Timing



- Things to note:
  - Timing is not in ticks
  - When Task2 unblocks, it “pre-empts” Task1; a context switch happens to transfers Task2 to running and Task1 to ready
  - Scheduler runs when Task2 blocks (call delay function) and unblocks (timer delay expires)

RTOS Pseudocode Example

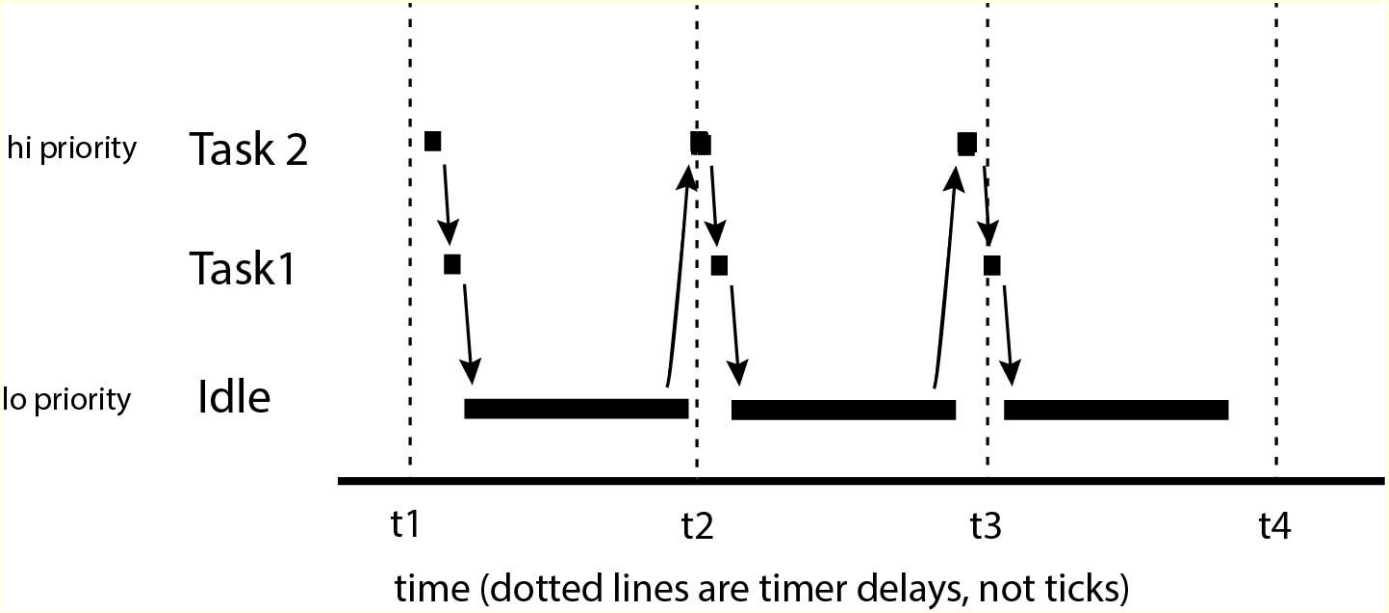
```
void Task1 (parameters) {
   for (;;) {
       Delay_API(#_ticks)         // code defining task
   }
}

void Task2 (parameters)
{
   for (;;) {
       Delay_API(#_ticks)         // code defining task
   }
}

int main (void)
{
   while (1)  {
    Create_task(Task1, priority=1); // lower priority task
    Create_task(Task2, priority=2);

    Start_scheduler();
   }
}
```

Example Timing



- Things to note:
  - Kernel operation (scheduler) is not shown
  - Task2 runs first, then blocks; Task1 runs next, the blocks
    - Blocking is initiated by call to Delay API
  - Idle task runs when Task2 and Task1 are in blocked states

Daily Terms and Definitions

- Pre-emption: when a higher priority task stops a lower priority task from running
- Idle Task: a task that runs when no other tasks are ready to run. The idle task is created by the RTOS and not the programmer
- Continuous processing task: A task that never blocks, which means it never enters the blocking state; such a task can only transition between ready and running states
- Starvation: when one task is unable to enter the running state because there is always a higher priority task ready to run